Arhitektura računara i operativni sistemi

Laboratorijske vežbe 4

1. Program i proces

Kada napišemo i kompajliramo kod, kao rezultat kompajliranja dobijamo fajl koji se naziva program ili izvršni fajl. Program u sebi sadrži same instrukcije, odnosno mašinski kod. Izvršni fajl će biti neaktivan sve dok ga ne izvršimo i iz tog razloga kažemo da je program pasivan entitet.

Pojam proces se u kontekstu operativnih sistema definiše na nekoliko načina:

- Proces je program u izvršavanju
- Proces je instanca programa koji je pokrenut na računaru
- Proces je entitet kojem može biti dodeljen procesor i drugi resursi računara
- Jedinica aktivnosti na računaru koju karakterišu izvršenje niza instrukcija, trenutno stanje i dodeljeni skup sistemskih resursa

Podaci o svakom procesu smeštaju se u strukturi podataka koja se naziva kontrolni blok procesa (engl. process control block), koju pravi i održava operativni sistem. Za kontrolni blok procesa bitno je da sadrži dovoljno podataka tako da je moguće prekinuti proces koji se izvršava i kasnije nastaviti izvršavanje kao da nije došlo do prekida. Kontrolni blok procesa je ključni alat koji omogućava operativnom sistemu da podrži višeprocesni način rada. Kada se proces prekine, trenutne vrednosti brojača naredbi i registara procesora (kontekstni podaci) čuvaju se u odgovarajućim poljima kontrolnog bloka procesa, a stanje procesa se menja u neku drugu vrednost, kao što je blokiran ili spreman. Operativni sistem sada može da stavi neki drugi proces u stanje izvršavanja, pri čemu se brojač naredbi i kontekstni podaci za taj proces učitavaju u registre procesora čime počinje njegovo izvršavanje.

Bitni delovi procesa su kod programa (koji može da se deli među procesima istog programa) i memorijske lokacije. Za svaki proces se čuvaju određeni podaci u kontrolnom bloku procesa:

- Identifikator procesa (engl. Process ID) – broj koji jedinstveno identifikuje svaki proces
- Stanje procesa (engl. State)
- Prioritet (engl. Priority)

- Brojač naredbi (engl. Program counter) – adresa sledeće instrukcije u programu koja će se izvršiti
- Pokazivači na memoriju (engl. Memory pointers) – obuhvata pokazivače na programski kod i podatke koji se odnose na ovaj proces, kao i pokazivače na deljene memorijske blokove
- Kontekstni podaci (engl. Context data) – podaci koji su prisutni u registrima i koji se čuvaju u trenutku kada se procesor oduzme od procesa
- U/I statusne informacije (engl. I/O status information) – obuhvata neobrađene U/I zahteve, U/I uređaje koji su dodeljeni procesu, listu datoteka koje proces koristi, ...
- Accounting information – mogu sadržati količinu procesorskog vremena i vreme upotrebe, vremenska ograničenja, brojeve naloga, ...

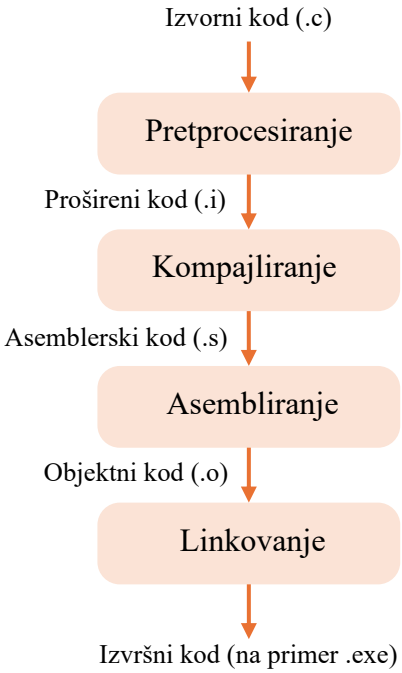
Za proces se još kaže i da je apstrakcija programa u izvršavanju. Za razliku od programa, proces je aktivan entitet i kratkog je životnog veka. Za razliku od programa kojem je od resursa računara potreban samo medijum za skladištenje podataka, proces za svoj rad zahteva centralnu procesorsku jedinicu, radnu memoriju i ulazno-izlazne uređaje. Ključne razlike između programa i procesa date su u sumarnoj tabeli:

Tabela 1 - Razlike između programa i procesa

Program	**Proces**
Pasivan entitet	Aktivan entitet
Potrebno mu je samo skladište	Potreban mu je CPU, radna memorija i U/I uređaji
Dug životni vek	Kratak životni vek

2. Proces prevođenja C koda u mašinski kod

Narednom šemom prikazan je niz koraka u procesu prevođenja C koda u mašinski kod:



Slika 1 - Proces prevođenja C koda u mašinski kod

2.1. Pretprocesiranje

Pretprocesiranje je prvi korak u fazi prevođenja C koda. Za ovaj korak odgovoran je program pod nazivom pretprocesor. U ovoj fazi dolazi do obrade pretprocesorskih direktiva. Pretprocesorske direktive u C-u počinju sa znakom **#**. Koraci u fazi pretprocesiranja jesu:

- Uklanjanje komentara

  Komentari u programskom jeziku C, kao i u bilo kom drugom jeziku, koriste se kao pojašnjenje ili napomena određenih delova koda. U fazi pretprocesiranja dolazi do uklanjanja komentara u kodu, s obzirom da nam oni nisu potrebni za prevođenje koda. Komentari u C-u označavaju se sa **`//`** u slučaju jednolinijskih komentara ili sa **`/* */`** u slučaju višelinijskih komentara.
- Proširivanje makroa (engl. macro expansion)

  Makroi su konstantne vrednosti ili izrazi koji se definišu pomoću **`#define`** direktive. Sintaksa za definisanje makroa je sledeća:

```
#define naziv_makroa vrednost_makroa
```

U fazi proširivanja makroa, gde god da se u kodu pojavi naziv_makroa, on biva zamenjen sa vrednošću tog makroa.

- Uključivanje fajlova (engl. file inclusion) Direktiva za uključivanje fajlova u C-u jeste

```
#include <naziv_fajla>
```

direktiva. Ova direktiva će u fazi pretprocesiranja biti zamenjena sadržajem fajla naziv_fajla.

- Uslovno kompajliranje (engl. conditional compilation)

  Delovi programa mogu da se kompajliraju uslovno koristeći odgovarajuću direktivu:

```
#if uslov
#ifdef identifikator
#ifndef identifikator
#elif uslov
#else
```

- Svaka od direktiva se evaluira, pri čemu se kod unutar direktiva koje nisu ispunjene odbacuje. Kada se naiđe na direktivu koja se evaluira na vrednost true, ostale direktive tipa **`#elif`** i **`#else`**, kao i kod unutar njih se odbacuju.

Kao izlaz iz faze pretprocesiranja dobija se prošireni C kod (sa ekstenzijom .i) koji će se koristiti u narednoj fazi.

2.2. Kompajliranje

U fazi kompajliranja, program koji se naziva kompajler prevodi prošireni .i kod u asemblerski kod (.s fajl). Asembler nije jedan jezik, već naziv za mnoštvo jezika, pri čemu za svaku procesorsku arhitekturu postoji određeni asemblerski jezik (engl. assembly). Faza kompajliranja je detaljnjije objašnjena u nastavku (3. Faze kompajliranja programskih jezika).

2.3. Asembliranje

Asemblerski kod dobijen u fazi kompajliranja se asembliranjem prevodi u mašinski kod. Program odgovoran za ovaj proces naziva se asembler. S obzirom da svaka instrukcija u asembleru ima odgovarajuću instrukciju u mašinskom kodu, asembler prolazi instrukciju po instrukciju i mapira je u odgovarajući binarni zapis. Fajl koji se dobija procesom asembliranja

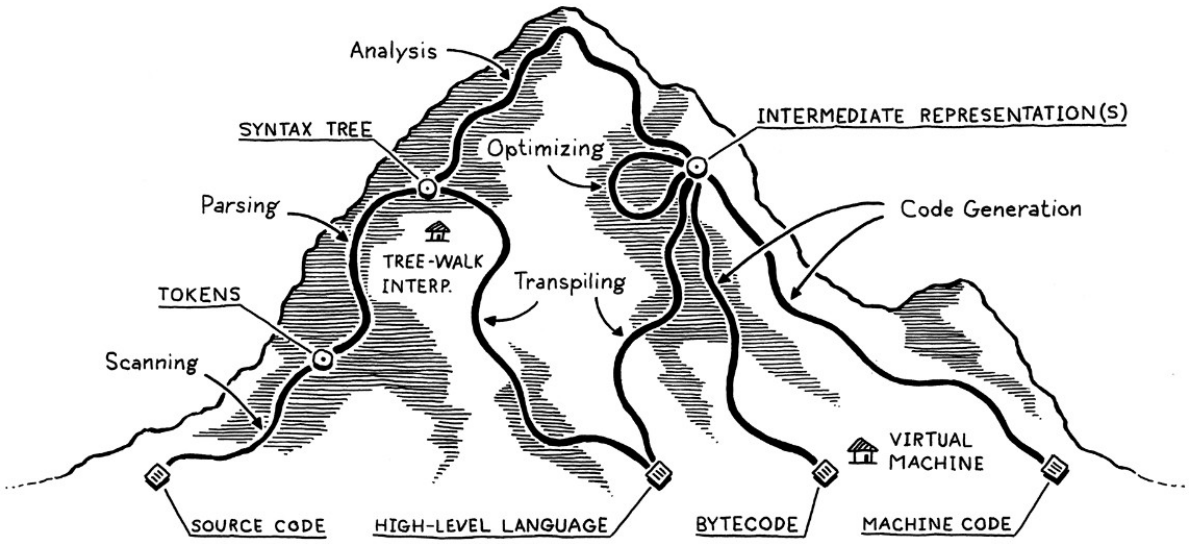
naziva se objektni fajl sa ekstenzijom .obj na sistemima zasnovanim na DOS-u ili ekstenzijom .o na UNIX sistemima.

2.4. Linkovanje

Linkovanje je proces tokom kojeg se spoljne biblioteke povezuju sa našim programom. Korišćenjem biblioteka postižemo pre svega uštedu vremena prilikom kucanja koda pošto ne moramo da ponovo implementiramo određene funkcionalnosti ukoliko one već postoje u okviru neke biblioteke. Kao izlaz iz faze linkovanja dobijamo izvršni fajl koji je moguće pokrenuti na sistemu.

3. Faze kompajliranja programskih jezika

Za implementaciju kompajlera moguće je koristiti različite metode i tehnike. Ipak, moguće je objediniti sve njih kroz jedan ilustrativni prikaz. Naime, faze kompajliranja možemo poistovetiti sa planinarenjem. Počinjemo od podnožja planine gde imamo izvorni kod našeg programa. Svaka od faza analizira naš program i transformiše ga u oblik u kojem će semantika programa (ono što želimo da program radi) biti jasnija. Kada dostignemo vrh planine možemo zapravo da uvidimo značenje samog koda. Ovde započinje spuštanje niz planinu. Dolazi do tranformisanja prikaza višeg nivoa u oblike nižeg nivoa da bismo na kraju došli do oblika koji naš računar razume.



Slika 2 - Ilustracija faza kompajliranja

3.1. Skeniranje

Prvi korak jeste skeniranje (engl. scanning), poznato i pod nazivom leksiranje ili leksička analiza. Skener je program koji prihvata niz karaktera kao ulaz, a kao izlaz vraća nešto nalik „rečima“. U terminologiji programskih jezika, reči se nazivaju tokeni.

Tako na primer, za liniju koda

```
int prosek = (min + max) / 2;
```

skener će vratiti sledeće tokene (označene sivom bojom):

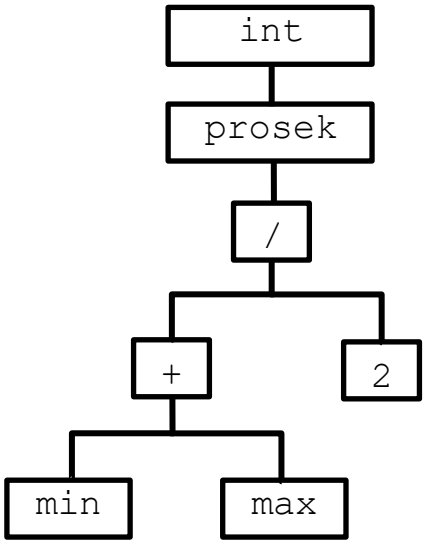
```
int prosek = ( min + max ) / 2 ;
```

U fazi skeniranja se najčešće zanemaruju razmaci i komentari s obzirom da oni obično nemaju nikakvo značenje.

3.2. Parsiranje

Sledeći faza je parsiranje. Kao što svaki govorni jezik ima gramatiku koja definiše pravila upotrebe jezika, tako i programski jezik poseduje svoju gramatiku. Gramatika programskog jezika omogućava nam da od tokena dobijenih u fazi skeniranja formiramo kompleksnije naredbe ili izraze.

Parser je program koji na osnovu niza tokena formira strukturu stabla koja replicira hijerarhijsku prirodu gramatike. Ovo stablo naziva se parsno stablo (engl. abstract syntax tree).



Slika 3 - Parsno stablo gorenavedene linije koda

Parsiranje je u prošlosti svoju primenu pronalazilo i u veštačkoj inteligenciji gde je pokušano da se parsira govorni jezik kako bi računari mogli da „razgovaraju“ sa nama. Ljudski jezik se ipak pokazao kao previše kompleksan da bi se prikazao preko parsnog stabla.

Još jedan od zadataka parsera jeste izveštavanje o sintaksnim greškama u kodu.

3.3. Semantička analiza

Prethodne dve faze koje su obrađene su dosta slične u većini implementacija. Od faze statičke analize uzimaju se u razmatranje pojedinosti jezika.

Prvi korak u semantičkoj analizi je tzv. vezivanje (engl. binding). Za svaki simbol pronalazimo gde je u kodu taj simbol definisan. Bitan pojma ovde jeste i oblast vidljivost (engl. scope) promenljive – oblast koda u kojem određeni simbol odgovara određenoj deklaraciji.

Ukoliko je jezik statički tipiziran u okviru semantičke analize se obavlja i provera tipova. Na primer, ukoliko imamo dve promenljive različitih tipova nad kojima primenjujemo binarnu operaciju koja nije podržana za ove tipove, semantička analiza će da vrati grešku u tipovima (engl. type error).

Podatke koje smo prikupili u toku semantičke analize moramo opet da smestimo u neku strukturu podataka. Postoje tri česta pristupa:

- Da modifikujemo parsno stablo tako što ćemo podatke dodati kao atribute čvorova stabla.
- Da koristimo tabelu čiji će ključevi biti nazivi simbola, a njihove vrednosti ono na šta se ti simboli odnose. Ovakva tabela naziva se tabela simbola.
- Da transformišemo parsno stablo u potpuno novu strukturu podataka koja će na bolji način prikazati semantiku koda. Ovaj pristup objašnjen je u sledećoj fazi.

Dosadašnje faze se predstavljaju front-end deo implementacije. Pre nego što pređemo na back-end, potrebno je da odradimo middle-end deo kojeg čine naredne dve faze.

3.4. Međujezik

Recimo da hoćemo da implementiramo kompajlere za jezike C, Pascal i Fortran koji će da prevode kod u x86, ARM i MIPS arhitekturu. To bi značilo da je potrebno da napišemo ukupno devet kompajlera:

C → x86, C → ARM, C → MIPS, Pascal → x86, Pascal → ARM, Pascal → MIPS, Fortran → x86, Fortran → ARM i Fortran → MIPS.

Potreban broj kompajlera možemo znatno da smanjimo upotrebom međujezika (engl. intermediate representation). Ukoliko koristimo ovaj pristup, potrebno je samo da napišemo po jednu front-end implementaciju za svaki od viših jezika i po jednu back-end implementaciju za svaku od arhitektura. Ovo je upravo način na koji i GCC kompajler podržava veliki broj arhitektura.

3.5. Optimizacija

Optimizacija predstavlja važan korak u procesu kompajliranja. Različiti kompajleri pristupaju različito ovoj fazi. Tako se, na primer, za programski jezik Lua generiše relativno neoptimizovan

kod, a većina napora za poboljšanje performansi se ulaže u toku izvršavanja samog koda.

Primer optimizacije jeste evaluacija konstanti (engl. constant folding). Ukoliko se određeni izraz uvek evaluira na istu vrednost, možemo tu evaluaciju izvršiti u toku kompajliranja programa čime bismo izraz zamenili krajnjim rezultatom.

Ukoliko, na primer, imamo sledeću liniju koda

```
povrsina = 3.14159 * (0.75 / 2) * (0.75 / 2);
```

metodom evaluacije konstanti možemo ga u fazi kompajliranja svesti na sledeću liniju

```
povrsina = 0.4417860938;
```

3.6. Generisanje koda

Poslednji korak jeste prevođenje koda u oblik koji mašina može da razume. Faza generisanja koda najčešće podrazumeva generisanje asemblerskog koda za konkretnu arhitekturu. Ova faza predstavlja back-end deo kompajlera. Ovde je potrebno doneti još jednu bitnu odluku. Da li da generišemo instrukcije za stvarnu ili virtuelnu arhitekturu.

Ukoliko generišemo kod za stvarnu arhitekturu, dobijamo izvršni fajl čije je instrukcije moguće direktno izvršiti na procesoru. Ovaj način daće najbolje moguće performanse, međutim generisanje mašinskog koda zahteva dosta posla. Još jedan nedostatak ove metode jeste što se vezujemo za specifičnu arhitekturu. Ukoliko, recimo, generišemo kod za x86 arhitekturu, nećemo ga moći pokrenuti na ARM arhitekturi.

Drugi način rešava probleme prethodnog načina, ali uz određene kompromise. Kompajler generiše kod za virtuelnu mašinu. Umesto instrukcija za stvarnu arhitekturu, dobijaju se instrukcije hipotetičke, idealne mašine. Niklaus Virt, tvorac programskog jezika Pascal, nazvao

je ovaj kod p kod skraćeno od portable, dok se danas najčešće sreće pod terminom bytecode, jer je svaka instrukcija najčešće veličine jednog bajta.

3.7. Virtuelna mašina

S obzirom na to da nijedna arhitektura ne podržava bytecode, ovaj kod je potrebno prevesti. Ovde opet imamo dve opcije.

Jedna opcija je da napišemo mini-kompajler za svaku od arhitektura, tako da bi se u ovom slučaju bytecode ponašao kao međujezik.

Druga opcija je da napišemo virtuelnu mašinu, program koji će da emulira hipotetičku arhitekturu i interpretira naš bytecode. Pokretanje bytecode-a uz pomoć virtuelne mašine je sporije od prevođenja u mašinski kod za konkretnu arhitekturu unapred, pošto se svaka instrukcija simulira u toku izvršavanja programa. Zauzvrat, dobijamo jednostavnost u implementaciji i portabilnost koda. Ukoliko implementiramo virtuelnu mašinu u, recimo, C-u, moguće je pokrenuti kod pisan u našem jeziku na bilo kojoj arhitekturi koja ima C kompajler.

Potrebno je napomenuti da termin virtuelna mašina odgovara različitim vrstama apstrakcije. Tip virtuelnih mašina o kojima smo mi pričali jesu virtuelne mašine za programske jezike (engl. language virtual machines) ili procesne virtuelne mašine. Pored njih bitno je pomenuti i sistemske virtuelne mašine. Sistemska virtuelna mašina (virtuelni računar) emulira čitavu hardversku platformu i operativni sistem. Na taj način je, recimo, moguće emulirati Windows sistem u okviru Linux-a i obrnuto. Pomoću sistemskih virtuelnih mašina cloud provajderi pružaju korisnicima iskustvo upravljanja njihovim „serverom“ bez potrebe da se fizički alociraju računari za svakog korisnika.

3.8. Sistem izvršavanja

Nakon svih dosadašnjih faza, dobijamo program u formi u kojoj možemo da ga izvršimo. Ukoliko smo kompajlirali kod u mašinski kod, potrebno je samo „reći“ operativnom sistemu da učita izvršni fajl. Ukolko smo kompajlirali kod u bytecode, potrebno je da pokrenemo virtuelnu mašinu na koju ćemo da učitamo naš program.

U oba slučaja, obično su nam za vreme izvršavanja programa potrebni neki dodatni servisi koje jezik nudi. Na primer, ukoliko jezik automatski upravlja memorijom, potreban je sakupljač smeća (engl. garbage collector) koji će da oslobodi memorijski prostor koji nije više u upotrebi.

Ove stvari dešavaju se za vreme izvršavanja programa za šta je odgovoran sistem izvršavanja (engl. runtime). Kod kompajliranih jezika, kod koji implementira sistem izvršavanja se injektuje u sam izvršni fajl. Kod interpretiranih jezika, sistem izvršavanja nalazi se u okviru virtuelne mašine.

4. Kompajliranje C koda upotrebom GCC kompajlera

GCC (GNU Compiler Collection) je skup kompajlera za razne programske jezike, i deo GNU porodice alata. Podržava više platformi i arhitektura od bilo kojeg drugog kompajlera, pa je tako dostupan i na brojnim ugrađenim (engl. embedded) sistemima, uključujući ARM i Power ISA arhitekture. GCC je prihvaćen kao standardni kompajler većine modernih UNIX sistema, što uključuje većinu Linux distribucija.

Da bismo iskompajlirali C kod pomoću GCC kompajlera koristimo sledeću naredbu:

```
gcc –o naziv_izvršnog_fajla putanja_do_izvornog_koda
```

kojoj prosleđujemo putanju do izvornog koda programa, kao i opciju –o kojom definišemo naziv izvršnog fajla koji će nam kompajler napraviti.

GCC među svojim alatima zapravo sadrži sve programe potrebne za prevođenje C koda u izvršni fajl: pretprocesor, kompajler, asembler i linker. U nastavku su date opcije koje je moguće proslediti naredbi gcc kako bismo dobili rezultat svake od faze prevođenja:

`gcc –E putanja_do_izvornog_koda` – vraća nam rezultat pretprocesiranja, odnosno prošireni C kod

`gcc –S putanja_do_izvornog_koda` – vraća nam rezultat kompajliranja, odnosno kod u asembleru

`gcc –c putanja_do_izvornog_koda` – vraća nam rezultat asembliranja, odnosno objektni fajl

`gcc –o naziv_izvršnog_fajla putanja_do_izvornog_koda` – vraća nam rezultat linkovanja, odnosno izvršni fajl.

5. Argumenti komandne linije u C-u

Programski jezik C nam, kao i brojni drugi jezici, daje mogućnost da iz komandne linije prosledimo vrednosti našem programu. Ove vrednosti nazivaju se argumenti komandne linije.

Argumentima komandne linije možemo pristupiti putem parametara main funkcije. Potrebno je da definišemo main funkciju sa dva parametra:

```
int main(int argc, char *argv[]) { /* ... */ }
```

Slika 4 - main funkcija sa parametrima za argumente komandne linije

Parametar **`argc`** je tipa int i sadrži podatak o broju argumenata komandne linije koje je korisnik prosledio. Bitno je napomenuti da se kao prvi argument uvek posmatra sam naziv programa, odnosno instrukcija koja je porenula program.

Parametar **`argv`** je niz niski (string-ova) i zapravo predstavlja niz argumenata koji su prosleđeni programu. Vrednost elementa **`argv[0]`** jeste instrukcija za pokretanje programa.

Neka je dat sledeći C kod:

```
#include <stdio.h>

int main(int argc, char *argv[]){

    printf("Ukupan broj argumenata je: %d\n", argc);

    for(int i = 0; i < argc; i++){
        printf("Vrednost argumenta %d je:%s\n", i, argv[i]);
    }

    return 0;
}
```

Slika 5 - Program za ispis argumenata komandne linije

koji prvo ispisuje broj argumenata komandne linije, a nakon toga iterira kroz sve argumente i ispisuje njihove vrednosti. Ukoliko iskompajliramo dati kod i pokrenemo ga naredbom

```
./argumenti AROS Linux
```

Kao rezultat izvršavanja programa dobijamo sledeći ispis:

```
Ukupan broj argumenata je: 3
Vrednost argumenta 0 je:./argumenti
Vrednost argumenta 1 je:AROS
Vrednost argumenta 2 je:Linux
```

Slika 6 - Izlaz programa argumenti

6. Pokazivačke funkcije

Pokazivačka funkcija je promenljiva koja sadrži adresu funkcije koja se može posle pozvati uz pomoć te promenljive.

Tipičan primer upotrebe pokazivačkih funkcija jesu funkcije kao argumenti drugih funkcija. Recimo da pišemo funkciju koja treba da sortira niz koji joj prosledimo i želimo da omogućimo da se pri pozivu funkcije definiše redosled kojim treba sortirati date vrednosti. Jedna opcija je da kao argument funkcije prosledimo jednu od predefinisanih vrednosti za redosled (npr. ascending za rastući ili descending za opadajući redosled). Međutim, ovaj metod nije fleksibilan s obzirom da je funkciji moguće proslediti samo redosled koji je već definisan od strane programera funkcije. Bolji način jeste da dozvolimo korisniku da sam definiše kriterijum za sortiranje vrednosti tako što će funkciji za sortiranje kao argument proslediti funkciju za poređenje vrednosti.

Sintaksa za deklarisanje pokazivačke funkcije u C-u je sledeća:

```
tip_povratne_vrednosti (*naziv_pok_funk)(tip_param1, tip_param2,
…)
```

Gde je sa tip_parami označen tip i-tog parametra funkcije na koju pokazivačka funkcija pokazuje. Pošto je u C-u naziv funkcije ništa drugo do adresa te funkcije, inicijalizacija pokazivačke funkcije vrši se na sledeći način:

```
naziv_pok_funk = naziv_funkcije;
```

Poziv funkcije pomoću pokazivačke funkcije ostvaruje se na sledeći način:

```
(*naziv_pok_funk)(argument1, argument2, …);
```

ili

```
naziv_pok_funk(argument1, argument2, …);
```

Upotreba pokazivačkih funkcija prikazana je u sledećem primeru:

```c
#include <stdio.h>

void funkcija(int x)
{
    printf("Vrednost prosleđenog argumenta je %d\n", x);
}

int main()
{
    void (*fp)(int) = funkcija;

    (*fp)(10);

    return 0;
}
```

Slika 7 - Upotreba pokazivačkih funkcija u C-u

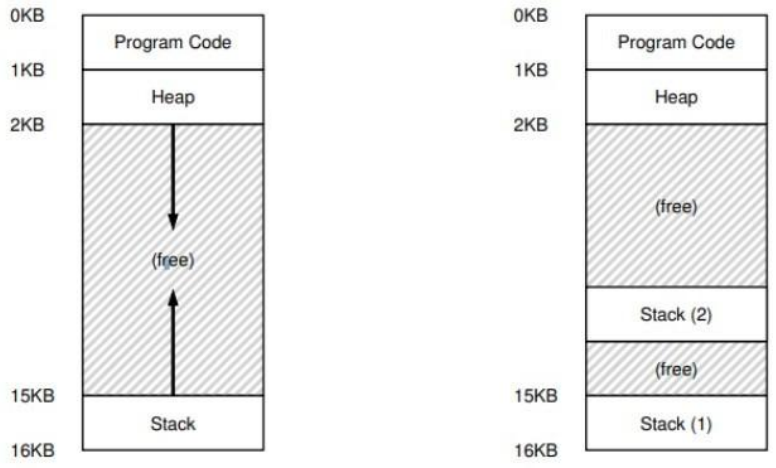
U main funkciji programa inicijalizuje se pokazivačka funkcija **`fp`** na adresu funkcije funkcija i nakon toga se poziva ova funkcija uz pomoć pokazivačke funkcije, pri čemu se prosleđuje argument **`10`**.

7. Niti

Nit (engl. thread) se u literaturi često definiše kao laki proces (engl. lightweight process). Do sada smo proces posmatrali kroz jednu tačku izvršavanja, odnosno svaki proces sadrži sopstvenu vrednost brojača naredbi pri čemu se instrukcije programa u okviru procesa izvršavaju sekvencijalno. Višenitni programi se sastoje od više tačaka izvršavanja, odnosno u okviru jednog procesa imamo više niti koje izvršavaju različite zadatke i svaka nit ima sopstvene vrednosti brojača naredbi i ostalih registara. Bitna razlika između niti i procesa jeste ta što za razliku od procesa gde svaki proces ima svoj adresni prostor, više niti koje pripadaju jednom procesu dele iste adresni prostor, pri čemu svaka nit ima poseban stek. Svaka nit, slično kao i kod procesa, čuva određene vrednosti u strukturi

koja se naziva kontrolni blok niti. Ovde se između ostalog čuvaju vrednosti brojača naredbi, vrednosti registara kao i stanje u kojem se nit nalazi. U jednoprocesorskom sistemu u kojem procesor u jednom trenutku može svoje resurse da dodeli samo jednoj niti prilikom dodele resursa niti dolazi do takozvanog context switching-a. Tokom ovog procesa, stanje registara jedne niti se čuva u kontrolni blok niti, dok se vrednosti registara druge niti učitavaju. Postoji jedna bitna razlika u odnosu na context switching koji se dešava kod procesa – kod niti adresni prostor ostaje nepromenjen, odnosno nema potrebe za menjanjem tabele stranica.

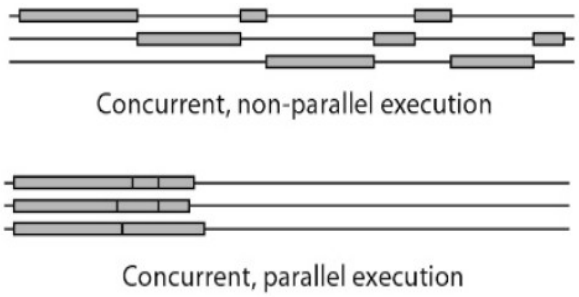
Adresni prostor jednonitnog i višenitnog programa dat je na sledećoj slici.



Slika 8 - Adresni prostor jednonitnog (levo) i višenitnog (desno) programa

8. Konkurentnost

U jednoprocesorskom multiprogramiranom sistemu, procesi se prepliću u vremenu tako da mogu da ostvare utisak istovremenog izvršavanja. Iako se ne postiže paralelna obrada, izvršavanje sa preklapanjem pruža benefite u pogledu efikasnosti obrade i strukturiranju programa. U višeprocesorskom sistemu, pored preplitanja izvršavanja više procesa, oni se mogu i preklapati. Ovo je ilustrovano na slici ispod.



Slika 9 - Konkurentno izvršavanje kod jednoprocesorskih (gore) i višeprocesorskih (dole) sistema

Kod jednoprocesorskih sistema, konkurentna obrada ostvaruje se upravo context switching-om gde je svaka nit (ili proces) odgovorna za jedan zadatak, a operativni sistem određuje koja nit će u kom trenutku da dobije resurse potrebne za njen rad.